IloveVI

Map Editor - Functional Analysis

Version 0.1 (draft for review) | 21 October 2026 | aligned with Functional Analysis v0.4 of the app

Status: draft. This document is the input for the implementation of the module tools:map-editor. The interactive prototype "IloveVI Map Editor" (design canvas) shows the screens described here; where the two differ, this document wins.

1. Purpose and scope

The Map Editor is an internal desktop tool (Compose for Desktop, module tools:map-editor). It lets one administrator place as many pins as needed on the game map, create the catalog entries (weapons, vehicles, items) together with their pins, review everything in a draft, and save it all to the database with a single Commit button.

In scope

- Sign in as an administrator and keep the session in memory.
- Show the game map (tiles from the backend, or a local PNG in offline mode) with pan and zoom.
- Place, select, move and delete pins; choose category and sub-type; undo and redo.
- Link each pin to an existing entry, or create a new entry with its pin; optional image.
- Keep an autosaved local draft; Commit it atomically; handle failures without losing work.

Out of scope (this version)

- Editing or deleting pins and entries that are already in the database (they are shown read-only). Planned for a later version.
- Generating tiles and registering a new map version (tools:tilegen, a separate tool).
- Songs, stations, news feeds and user management.
- Any user position or location feature: none exists in this product.
- A mobile version: the tool is for desktop only.

2. Users and preconditions

Item	Description
Role	One role only: administrator. The token must carry app_metadata role = admin; the database policies accept writes from no one else.
Environment	SUPABASE_URL and SUPABASE_ANON_KEY (the publishable key). The service_role key is never used or stored by the tool.
Backend	Migrations applied (entries, map_config, map_pins, storage buckets) and the function commit_map_edits created.
Map	A map_config row (map version, image size, tile template) or a local PNG opened with the offline mode.
Network	Needed for sign-in, for reading entries and pins, and for Commit. Placing pins and editing the draft work without it.

3. Concepts

Term	Meaning
Entry	A catalog record (weapon, vehicle or item) with name, sub-type, zone, description, optional attributes and image. One entry can have many pins.
Pin	A point on the map linked to one entry, with normalized coordinates x and y between 0 and 1 and the map version it was placed on.
Draft	The set of new pins and new entries not yet saved to the database. It lives in memory and in a local file (draft.json).
Database pin	A pin already saved. Drawn dimmed and read-only.
Incomplete pin	A draft pin that has no entry yet, or whose new entry has no name. It blocks Commit.
Active entry	The entry used for every new click in Place mode ("Place another pin of this entry").
Commit	One atomic save of all draft entries, images and pins. All or nothing.

4. Window layout

The window has a fixed minimum size of 1280 x 800 and is laid out as in the schematic below (1440 x 900 shown).

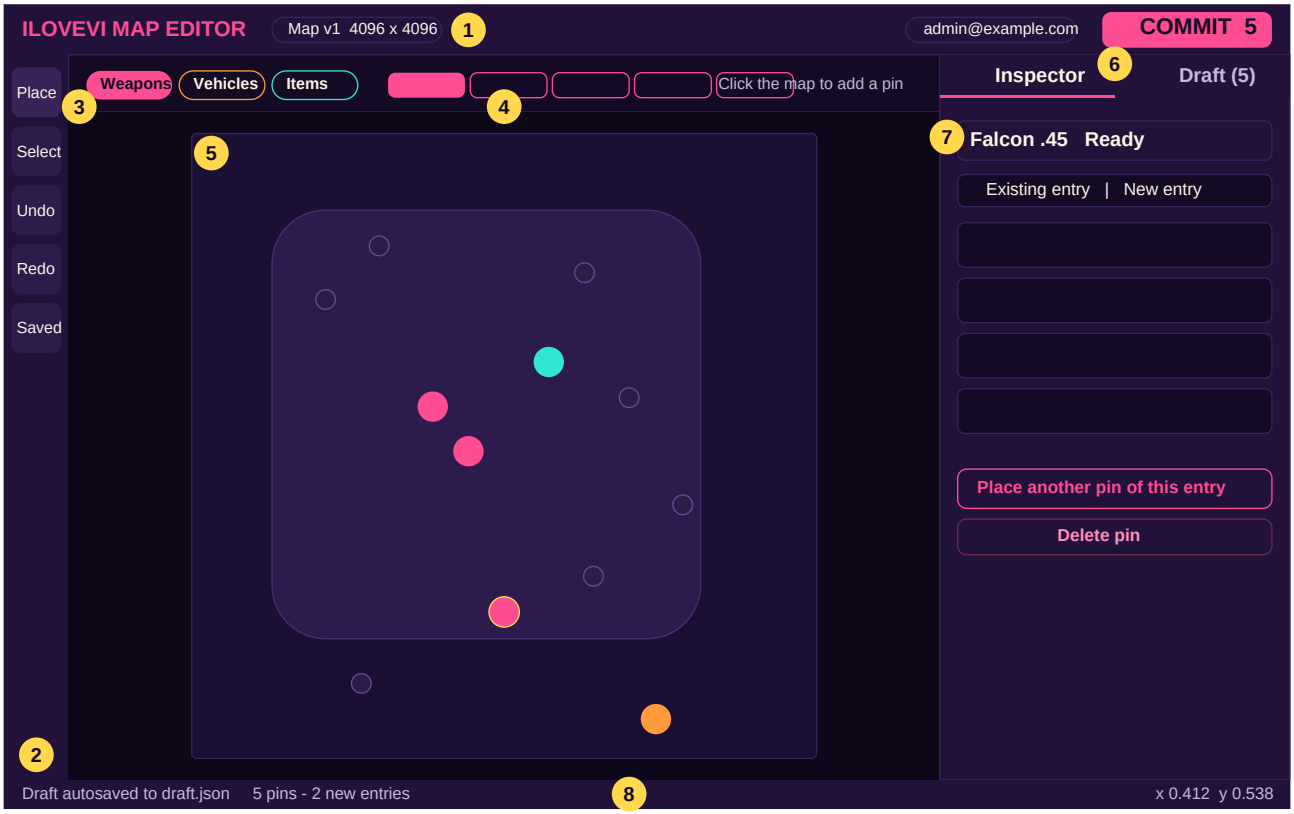

Figure 1 - Layout schematic. The interactive prototype has the exact look.

#	Region	Content
1	Header (56 px)	Wordmark, map version chip (from map_config), warning "N pins need an entry" when relevant, signed-in user chip, Commit button with the number of draft pins.
2	Tool column (72 px)	Place, Select, Undo, Redo, and a toggle to show or hide the pins already in the database.
3	Category row (64 px)	Category chips (Weapons, Vehicles, Items) and sub-type chips for new pins; the active-entry banner replaces the sub-type chips when an entry is active; a short hint on the right.
4	Sub-type chips	Sub-types of the chosen category, each with its icon (13 icons of the design system).
5	Map	The map with draft pins (category color, sub-type icon, selected ring, yellow warning badge when incomplete) and dimmed database pins.
6	Right panel tabs (392 px wide)	Inspector and Draft (n).
7	Inspector	Header of the selected pin, entry source switch, picker or form, actions.
8	Status bar (32 px)	Autosave status, counts of pins and new entries, cursor coordinates over the map.

Visual style

Same design tokens as the app (Night theme): background #140A24, panels #22123A, borders #3A2358, text #FFF4E8, secondary text #C9BADB, pink #FF4D94 for weapons, orange #FF9A3D for vehicles, teal #2FE6D2 for items, yellow #FFD84D for warnings; fonts Anton for titles and the Commit button, Barlow for everything else. Components and icons come from core:designsystem; nothing is redrawn locally.

5. Functional requirements

Priorities: Must = needed for the first use, Should = expected, Could = if time allows.

5.1 Session

ID	Requirement	Priority
EDT-01	A sign-in dialog asks for email and password and signs in with Supabase Auth using SUPABASE_URL and SUPABASE_ANON_KEY. The session is kept in memory only.	Must
EDT-02	If the account has no admin role, show a clear message, allow looking at the map and preparing a draft, and keep Commit disabled.	Must
EDT-03	The header shows the signed-in user and the map version; a sign-out action returns to the sign-in dialog and keeps the draft.	Must
EDT-04	If the token expires during the session, ask to sign in again without losing the draft.	Should

5.2 Map

ID	Requirement	Priority
EDT-10	Show the map with pan, pinch or scroll zoom and double-click zoom, using the shared viewer of core:mapview (tiles or a single local image).	Must
EDT-11	Pins already in the database are shown dimmed and read-only. A tool-column toggle shows or hides them.	Must
EDT-12	The status bar shows the normalized coordinates (x, y between 0 and 1, three decimals) under the cursor.	Should
EDT-13	Every position is stored as normalized coordinates, never in pixels, so pins stay right at every zoom and size.	Must
EDT-14	If map_config is missing, show an explanation and a button to open a local PNG in offline mode.	Should

5.3 Modes and toolbar

ID	Requirement	Priority
EDT-20	Two modes: Place and Select. The active one is highlighted. Place shows a crosshair cursor.	Must
EDT-21	Category chips and sub-type chips choose the category and sub-type of the pins about to be placed.	Must
EDT-22	Undo and redo for placing, moving, deleting and linking pins to entries: at least 50 steps. They are disabled when there is nothing to undo or redo.	Must
EDT-23	Keyboard: Delete removes the selected pin; Ctrl or Cmd Z and Shift Z undo and redo; Escape deselects.	Should

5.4 Placing pins

ID	Requirement	Priority
EDT-30	In Place mode a click adds a pin of the chosen category and sub-type at that position. Any number of pins can be added in a row.	Must
EDT-31	The new pin is selected and the Inspector opens, unless an entry is active.	Must
EDT-32	"Place another pin of this entry" makes the entry active: every click adds a pin of that entry. A banner above the map names it and has a close button to stop.	Must
EDT-33	A draft pin uses the category color and the sub-type icon, a ring when selected, and a yellow warning badge when incomplete.	Must

5.5 Selecting and editing pins

ID	Requirement	Priority
EDT-40	In Select mode a click selects a pin; dragging moves it; clicking an empty area of the map deselects.	Must
EDT-41	A Delete pin action removes the selected draft pin. A new entry used only by that pin is removed with it.	Must
EDT-42	Select several pins with a rectangle or Shift-click, then delete them or change their category together.	Could

5.6 Inspector and entries

ID	Requirement	Priority
EDT-50	The Inspector header shows the pin icon, the entry name (or "New ... pin"), the coordinates and sub-type, and a badge: "Needs an entry" (yellow) or "Ready" (teal).	Must
EDT-51	A switch chooses the entry source: Existing entry or New entry.	Must
EDT-52	Existing entry: a search box (name or zone) filters the entries of the pin's category; choosing one links the pin. With no match, a hint suggests creating a new entry.	Must
EDT-53	A pin linked to an existing entry shows its name, sub-type and zone, and a Change entry action.	Must
EDT-54	New entry form: name (required), sub-type, zone, description, the extra fields of the category, and an optional image. The fields come from the real columns of the entry tables, defined in one place.	Must
EDT-55	Duplicate guard: if a new entry has the same name and category as an existing one, show a warning with a button to use the existing entry.	Should
EDT-56	When an entry is used by several draft pins, the Inspector says how many.	Should
EDT-57	For a pin with no entry, the category can be changed in the Inspector (the sub-type chips follow).	Should
EDT-58	Switching a pin from New entry back to Existing entry discards the unused new entry.	Must

5.7 Draft

ID	Requirement	Priority
EDT-60	The Draft tab shows the number of pins per category and the list of draft pins grouped by category, each with icon, entry name (or "No entry yet" in yellow), sub-type and coordinates. Clicking a row selects the pin.	Must
EDT-61	The draft is saved to a local JSON file after every change and reloaded when the tool starts. Every pin and entry has a client-generated UUID.	Must
EDT-62	Discard draft needs two clicks (the button changes to "Click again to discard the whole draft"). Discarding is undoable until the tool is closed.	Must

5.8 Commit

ID	Requirement	Priority
EDT-70	The Commit button is enabled only when the draft has changes, every pin is complete and the account is admin. It shows the number of draft pins. When some pins are incomplete, the header says how many.	Must
EDT-71	Pressing Commit opens a confirmation dialog with: new entries (and how many have an image), new pins by category, pins on existing entries, map version, signed-in user. Buttons Cancel and Commit.	Must
EDT-72	The save is atomic: entries, images and pins are saved together or not at all, through the database function commit_map_edits (section 7).	Must
EDT-73	While saving, the dialog shows progress and what is being saved. Commit cannot be pressed twice.	Must
EDT-74	On success: the dialog states how many entries, pins and images were saved; the draft is emptied; the saved pins become normal database pins on the map.	Must
EDT-75	On failure: the database is unchanged, the draft stays intact, the dialog shows the error and a Retry button.	Must
EDT-76	Retrying the same Commit never creates duplicates, because ids are generated on the client and saved with upsert.	Must
EDT-77	Before sending, validate: coordinates between 0 and 1, names not empty, sub-types valid for the category, map version equal to the current one. If the map version on the server changed since the draft was started, warn and stop.	Must

5.9 Images

ID	Requirement	Priority
EDT-80	The new entry form lets the user choose a local image (PNG, JPEG or WebP, 5 MB at most). It is shown with its file name and a remove button.	Should
EDT-81	Images are uploaded to the entry-images bucket inside the Commit flow and the public address is saved in imageUrl. If an upload fails the whole Commit fails and the draft is kept.	Should

6. States and edge cases

Situation	Expected behavior
Nothing selected (Inspector)	Dashed card "No pin selected" explaining how to place and select.
Empty draft	Draft tab shows zero counts and a dashed card; Commit disabled; Discard hidden.
Pins with no entry	Yellow badge on the pin and in the list; header warning; Commit disabled.
Offline	Placing and editing work; Commit shows a clear error and Retry; reading entries shows a banner.
Token expired	Sign-in dialog opens on top; draft untouched; Commit repeats after sign-in.
Not an admin	Message on sign-in; Commit disabled with an explanation.
Map version changed on the server	Warning at Commit time naming both versions; the user decides how to proceed.
Duplicate entry name	Warning in the form with a button to use the existing entry.
Image too large or wrong type	Inline error under the image field; nothing is attached.
Tool closed during Commit	On next start the draft is still there; retry is safe (EDT-76).
Draft file damaged	Start with an empty draft and keep a copy of the damaged file next to it.

7. Data and backend contract

The tool reads entries, pins and map_config with the publishable key, and writes only through the function below. Exact table and column names must be taken from the migrations of the repository, not from this document.

Item	Description
Function	public.commit_map_edits(payload jsonb) returns jsonb, in a new migration. Runs with the caller's rights; refuses anyone without the admin role claim.
Payload	mapVersion; entries[] (id, category, subType, name, zone, description, extra attributes, imageUrl); pins[] (id, category, entryId, x, y, zone).
Behavior	Validate everything first; upsert entries into the table of their category; upsert pins; follow the existing updated_at and soft-delete conventions; all in one transaction.
Result	Counts of entries and pins saved. Any error rolls back everything.
Storage	A bucket for entry images with public read and admin-only write, created in a migration (same policies as the tile bucket).
Idempotency	Same ids on a retry update the same rows.

8. Non-functional requirements

Area	Requirement
Platform	Compose for Desktop on macOS first; Windows and Linux should work with no extra code.
Performance	Smooth pan, zoom and placing with 500 draft pins and 2000 database pins.
Security	Only the publishable key; no service_role key; no password written to disk; session in memory.
Reliability	No loss of work: draft autosaved after every change; a failed Commit changes nothing.
Accessibility	Every control reachable by keyboard with a visible focus; icon-only buttons have text labels; contrast of 4.5:1 for text.
Consistency	Components, icons, colors and fonts from core:designsystem and core:mapview; the same module pattern as the other modules.
Language	Interface in English only.

9. Acceptance scenarios

- Place 50 pins of different categories with a few clicks; each shows the right color and icon.
- Create a new weapon with its pin, then use "Place another pin of this entry" to add four more pins of the same weapon; Commit saves one entry and five pins.
- Link a pin to an existing entry found by search; Commit saves only the pin.
- Leave one pin without an entry: Commit stays disabled and the header names the problem.
- Press Commit with the network off: error and Retry; the draft is intact; after reconnecting, Retry succeeds and creates no duplicates.
- Close the tool with a draft and reopen it: the draft is restored exactly.
- Sign in with a non-admin account: Commit stays disabled and the message explains why.
- Undo and redo restore pins, moves, deletions and entry links.
- After a successful Commit, the entries and pins are visible in the database tables and in the app, in the same place at every zoom.

10. Open questions

#	Topic	Question and proposal
1	Entry attributes	Which attributes does each category have? The prototype shows price and ammo type for weapons, price and seats for vehicles, none for items. Proposed: read them from the migrations and keep one definition in the code.
2	Zone of a pin	Is the zone stored on the pin, on the entry, or both? Proposed: the pin takes the zone of its entry by default and it can be overridden later.
3	Existing pins	When may the tool edit or delete saved pins? Proposed: next version, with a reposition list for pins placed on an older map version.
4	Image size	Maximum size and format of entry images. Proposed: 5 MB, PNG, JPEG or WebP, resized by the backend later.
5	Item types	The item sub-types (ammo, armor) are placeholders until the real GTA VI classification is known.